Newton or Lagrange for the quasi-steady trim: why the Lagrangian form is not faster

AWETrim notes

10 October 2026

Question. The reduced-order model writes the kite's dynamics as a force balance in the course-aligned reference frame (`SystemModel.force_residual`, $-m\boldsymbol{a}+\boldsymbol{F}_{\text{ext}}$; Cayon, van Deursen, Schmehl 2026, WES 11, 1097). Would deriving the same model from a Lagrangian make the trajectory NLP cheaper?

Answer. No. The Lagrangian derivation yields the same three equations, the inertia terms are 0.3 % of each node function, and function evaluations are a quarter of IPOPT's wall time. The numbers below come from `scripts/reduced-order-model/optimization/studies/compare_lagrangian_residual.py` (LEI-V3, periodic B-spline reel-out, $M = 10$, flight-corrected aerostructural ROM).

1 The two derivations

The path is prescribed on the unit sphere, $\boldsymbol{q}(s) = (\phi(s), \beta(s))$, and the kite sits at $\boldsymbol{R} = r\,\boldsymbol{q}(s)$. The course frame has x along the path tangent, z radially outward and y normal to the path on the sphere.

Newton (library). Velocity $\boldsymbol{v} = (v_\tau, 0, v_r)$ with $v_\tau = r\,\sigma'\,\dot{s}$, $\sigma' = |\boldsymbol{q}'|$, and the frame rotation rate

$$\boldsymbol{\omega} = \Big(0,\ \frac{v_\tau}{r},\ \frac{v_\tau}{r}\tan\beta\sin\chi - \dot{\chi}\Big). \tag{1}$$

The acceleration is $\boldsymbol{a} = (\dot{v}_\tau, 0, \dot{v}_r) + \boldsymbol{\omega}\times\boldsymbol{v}$; the quasi-steady model drops the first bracket. Three rows: $\boldsymbol{F}_{\text{ext}} - m\boldsymbol{a} = \boldsymbol{0}$.

Lagrange. Generalised coordinates (s, r), the path being a holonomic constraint. Kinetic energy $T = \frac{m}{2}\big(\dot{r}^2 + r^2\sigma'^2\dot{s}^2\big)$, generalised forces $Q_s = \boldsymbol{F}\cdot\partial\boldsymbol{R}/\partial s = r\sigma' F_x$ and $Q_r = F_z$. The Euler–Lagrange equations are

$$s:\quad \frac{d}{dt}\big(mr^2\sigma'^2\dot{s}\big) - mr^2\sigma'\sigma''\dot{s}^2 = r\sigma' F_x, \tag{2}$$

$$r:\quad m\big(\ddot{r} - r\sigma'^2\dot{s}^2\big) = F_z. \tag{3}$$

Expanding the s row gives $mr\sigma'\,(\dot{v}_\tau + v_\tau v_r/r)$, so the quasi-steady assumption $\dot{v}_\tau = 0$ is a condition on $\ddot{s}$, not $\ddot{s} = 0$. The lateral force the steering has to supply is the constraint multiplier of the path, $mk_g v_\tau^2/r$, with k_g the geodesic curvature. Divided by the metric factors, the three rows are

$$F_x - m\frac{v_\tau v_r}{r} = 0, \qquad F_y + mk_g\frac{v_\tau^2}{r} = 0, \qquad F_z + m\frac{v_\tau^2}{r} = 0, \tag{4}$$

which are the Newton rows. Over 200 random states the two residuals agree to 3.5×10^{-6} relative; the difference is the `fmax` floor on v_τ and the 10^{-6} guard in $\tan\beta$ of the library form. The multiplier row cannot be dropped, since it is the equation that determines u_s, so no equation and no unknown is eliminated.

2 Where the operations are

Table 1: SX instruction counts of the per-node trim residual.

expression	full spline	local support (NLP)
Newton residual	9971	3429
Lagrange residual	20585	6463
path kinematics $\phi, \beta, \chi, \dot{\chi}, v_\tau$	9195	
aerodynamic force (ROM)	9846	
inertia terms given the kinematics (Newton)	26	

The whole acceleration costs 8877 instructions, but 26 of them are the inertia; the rest is the spline evaluation and its s-derivatives, which the aerodynamic force needs as well (the apparent wind is built in the course frame). The Lagrangian residual, written through `curvature_geodesic`, is twice as large because the derivative chain of the spline is built a second time. The Jacobians scale the same way (13460 against 22074).

3 Where the time is

Reel-out NLP, 120 nodes, C_ϕ, C_β and the depower free, the production IPOPT options (`Phase.run_opti`: L-BFGS, MUMPS, adaptive barrier):

iterations	56
IPOPT wall time	1.4 s
function evaluations	0.36 s (26 %), 0.30 s of it the inequality Jacobian
MUMPS back-solves and barrier updates	74 %
inertia terms' share of the function evaluations	about 0.3 %, ∼1 ms

A reformulation of the inertia terms can therefore change at most a millisecond per solve. The levers that remain are the spline-kinematics graph in `ParametrizedKinematics` (nested `ca.gradient` chains, about 90 % of the node operations), the iteration count, and the KKT size.

4 Follow-up: the remaining levers, measured

Same 120-node reel-out NLP, single runs, so differences of a few milliseconds per iteration are noise.

Spline-kinematics graph. The properties of `ParametrizedKinematics` rebuild the spline sub-graph on every access (ϕ alone is evaluated about nine times inside one node function) and the SX expansion keeps the copies. Written with analytic B-spline derivatives, the course-frame chain $\chi, v_\tau, \dot{\chi}, k_g$ costs 470 instructions instead of 4951 (agreement 10^{-11}). The cheapest fix, caching the properties so each sub-expression is built once, gives most of that:

node function	properties	cached
residual	3429	2280
angle of attack	2868	1719
curvature triple	5626	2013

Yet the solve does not get faster: 56 iterations either way, 24 against 26 ms per iteration without turn-radius rows, 31 against 28 ms with them ($R_{\min} = 25$ m, 95 against 94 iterations). Function evaluations stay at 25 % of the wall time; the rest is the KKT back-solve, which L-BFGS turns into many right-hand sides per iteration. The smaller graph still cuts the whole-graph expansion memory in proportion, which matters at the full-cycle size (3.7 MB per node at $N = 376$), and would be a safe change: the mathematics is unchanged.

Hessian. The exact Hessian (available from the SX graph) halves the iterations, 26 instead of 56, but the Hessian evaluation costs 0.69 s and the solve ends at 1.66 s against 1.36 s. With the turn-radius rows it is far worse: 245 iterations and 29 s, the σ'^3 polynomial rows giving an ill-conditioned Hessian. A 20-deep L-BFGS memory changes nothing (57 and 84 iterations). The production choice, L-BFGS with the default memory, stands.

Conclusion. At this problem size the NLP is already balanced between function evaluation and linear algebra, and no single lever gains more than the 25 % that function evaluation represents. Wall time is governed by the iteration count and the number of staged passes, not by the cost of the model.

Reproduce. The A/B uses the flags `--turn-radius 25`, `--kinematics-cache` and `--exact-hessian` of

```
python scripts/reduced-order-model/optimization/studies/compare_lagrangian_residual.py --nlp
```